ISEN
ALL IS DIGITAL!
OUEST

TP 2 Data-mining : Arbres de décisions

Exercice 1

On considère le jeu de données suivant, généré artificiellement :

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

n = 50
np.random.seed(1234)
X = np.random.uniform(0, 1, n)

np.random.seed(5678)
# Generation de Y avec une relation non-lineaire par morceaux
Y = 1 * X * (X <= 0.6) + (-1 * X + 3.2) * (X > 0.6) + np.random.normal(0, 0.1, n)

data1 = pd.DataFrame({'X': X, 'Y': Y})

# Visualisation
plt.scatter(data1['X'], data1['Y'])
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

1. À l'aide de la classe `DecisionTreeRegressor` du module `sklearn.tree`, construire un arbre permettant d'expliquer Y par X. *Rappel : sous scikit-learn, la matrice des caractéristiques doit être en 2D. Vous pouvez utiliser* `data1[['X']]` *au lieu de* `data1['X']`.
2. Représenter l'arbre avec la fonction `plot_tree` de `sklearn.tree`.
3. Écrire l'estimateur mathématique associé à l'arbre.
4. Ajouter sur le graphe de la question 1 la partition définie par l'arbre ainsi que la courbe des valeurs prédites (la fonction en escalier).

Exercice 2

On considère maintenant un jeu de données bidimensionnel pour de la classification :

```python
n = 50
np.random.seed(12345)
X1 = np.random.uniform(0, 1, n)

np.random.seed(5678)
X2 = np.random.uniform(0, 1, n)

Y = np.zeros(n)

np.random.seed(54321)
```

```python
mask1 = (X1 <= 0.45)
Y[mask1] = np.random.binomial(1, 0.85, np.sum(mask1))

np.random.seed(52432)
mask2 = (X1 > 0.45)
Y[mask2] = np.random.binomial(1, 0.15, np.sum(mask2))

data2 = pd.DataFrame({'X1': X1, 'X2': X2, 'Y': Y})

# Visualisation (les couleurs representent les classes)
plt.scatter(data2['X1'], data2['X2'], c=data2['Y'], cmap='bwr', s=50)
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()
```

1. Construire un arbre de classification (via `DecisionTreeClassifier`) permettant d'expliquer Y par $X1$ et $X2$. Représenter l'arbre et identifier l'éventuel problème (observez la complexité et la profondeur de l'arbre généré par défaut).
2. Écrire la règle de classification ainsi que la fonction de score définies par l'arbre simplifié (après avoir par exemple limité sa profondeur avec `max_depth`).
3. Ajouter sur le graphe de la question 1 la partition définie par l'arbre (vous pourrez générer une grille de points avec `np.meshgrid` et prédire leurs classes pour tracer les frontières de décision avec `plt.contourf`).

Exercice 3

On considère un dernier jeu de données contenant une variable explicative catégorielle :

```python
n = 100
# On repete le motif ['A', 'B', 'C', 'D'] n fois
X = np.tile(['A', 'B', 'C', 'D'], n)
Y = np.zeros(len(X))

np.random.seed(1234)

maskA = (X == 'A')
Y[maskA] = np.random.binomial(1, 0.9, np.sum(maskA))

maskB = (X == 'B')
Y[maskB] = np.random.binomial(1, 0.25, np.sum(maskB))

maskC = (X == 'C')
Y[maskC] = np.random.binomial(1, 0.8, np.sum(maskC))

maskD = (X == 'D')
Y[maskD] = np.random.binomial(1, 0.2, np.sum(maskD))

data3 = pd.DataFrame({'X': X, 'Y': Y})
# Y doit etre traite comme une categorie (type entier ou chaine)
data3['Y'] = data3['Y'].astype(int)
```

1. Construire un arbre permettant d'expliquer Y par X. *Attention : contrairement à R, l'implémentation standard des arbres dans* ***`scikit-learn`*** *ne gère pas nativement les variables catégorielles non numériques. Il vous faudra d'abord transformer la variable X (par exemple avec la fonction* ***`pd.get_dummies`*** *pour appliquer un encodage One-Hot).*

2. Expliquer la manière dont l'arbre est construit dans ce cadre-là (comment scikit-learn effectue ses coupures sur les variables binaires issues du One-Hot encoding par rapport aux sous-ensembles de modalités natives gérées par R).